# Knowledge Organiser — 2.2 Problem Solving & Programming

*OCR A Level Computer Science H446 — Component 02 · 2.2.1 Programming techniques + 2.2.2 Computational methods. All code = OCR Exam Reference Language.*

## Programming constructs

| Construct | Meaning | Key point |
|---|---|---|
| **Sequence** | Instructions run one after another, in order | Order matters — swapping lines changes the result |
| **Selection / branching** | Choose a path based on a condition (`if` / `elseif` / `else`, `switch` / `case`) | — |
| **Count-controlled iteration** | Repeat a *fixed, known* number of times (`for...next`) | Number of repeats known in advance |
| **Condition-controlled iteration** | Repeat *until a condition changes* (`while`, `do...until`) | Number of repeats not known in advance |

- `while` = pre-conditioned (tests at **start**) → body may run **zero** times.
- `do...until` = post-conditioned (tests at **end**) → body always runs **at least once** (good for input validation).

## Recursion

- A subroutine that **calls itself**. Must have: **base case** (stops recursion, no further call) + **general/recursive case** (calls itself, moving closer to base case).
- Missing/unreached base case → endless calls → **stack overflow**.
- **Call stack:** each call **pushes a stack frame** storing parameters, local variables, and **return address**. Frames are **popped** in reverse (LIFO) as calls return.
- **Uses:** self-similar problems — tree traversal, factorial/Fibonacci, quicksort, merge sort, folder navigation, Towers of Hanoi.

| Aspect | Recursion | Iteration |
|---|---|---|
| Memory | More (a frame per call) | Less (one set of variables) |
| Speed | Slower (call overhead) | Faster |
| Code | Shorter/elegant for self-similar problems | Longer for tree-like problems |
| Risk | Stack overflow | Infinite loop |
| Best for | Tree/recursive structures | Linear, repeated tasks |

Any recursive solution can be rewritten iteratively (often with an explicit stack), and vice versa.

## Scope — local vs global

**Scope** = the region of the program where a variable can be accessed (is "visible").

| | **Local** | **Global** |
|---|---|---|
| Declared | Inside a subroutine | Outside any subroutine (main program) |
| Visible | Only within that subroutine | Anywhere in the program |
| Lifetime | While subroutine runs | Whole run |

- **Prefer local:** fewer side effects, frees memory on exit, more modular/reusable/easier to debug.
- **Globals** only for genuinely program-wide shared values; overuse harms maintainability. A local **shadows** a same-named global within its subroutine.

## Modularity, functions & procedures

- **Modularity** = splitting a program into small self-contained **subroutines (modules)**, each doing one task. Benefits: easier to write/test/debug, reusable, team can split work, easier to read/maintain.
- **Procedure** = performs a task, **returns no value** (call is a statement: `greet("Ada")`).
- **Function** = performs a task and **returns a single value** (usable in an expression: `y = square(4) + 1`).

## Parameters — by value vs by reference

- **Parameter** = variable in the subroutine *definition*; **argument** = actual value supplied at *call*.

| | **By value** | **By reference** |
|---|---|---|
| What is passed | A **copy** of the value | The **memory address** of the original |
| Effect on original | **Unchanged** | **Can be changed** |
| Advantage | Safer (no side effects) | Efficient for large data (no copy); can return changes |

## IDE features

- Editor with **syntax highlighting**; **code completion / intellisense**; **auto-indentation / pretty-printing**.
- **Error diagnostics/highlighting** (flags syntax errors as you type).
- **Debugging: breakpoints** (pause at a line), **stepping** (run one line at a time), **variable watch / watch window** (inspect values at run time).
- Built-in **translator (compiler/interpreter)** + run-time environment; **call stack viewer**.

## Object-oriented programming (OOP)

| Term | Definition |
|---|---|
| **Class** | Template/blueprint defining the attributes and methods of its objects |
| **Object** | A specific **instance** of a class, created at run time |
| **Attribute** (field) | A data variable belonging to an object |
| **Method** | A subroutine belonging to a class |
| **Constructor** | Special method that runs on creation to initialise attributes (OCR: `new`) |
| **Getter / Setter** | Public methods to read/write private attributes safely (setter can validate) |
| **Encapsulation** | Bundle data + methods in a class; keep attributes **private**, access via public methods (data hiding → protects integrity) |
| **Inheritance** | Subclass/child acquires attributes & methods of a superclass/parent (OCR: `inherits`); models "is-a", promotes reuse |
| **Polymorphism** | Same method call behaves differently per object type; usually via **overriding** an inherited method |

- **Overriding** = subclass replaces an inherited method (same name + parameters). **Overloading** = same name, different parameter lists.

## 2.2.2 Computational methods

**Problem solvable computationally if it:** can be **decomposed**; has clear **inputs/processes/outputs**; is repetitive/rule-based (algorithmic); allows useful **abstractions**; data can be **structured/modelled**; has a clear, **finite** sequence of steps. *(Some problems are non-computable e.g. Halting Problem; some computable but intractable.)*

| Approach | Meaning |
|---|---|
| **Problem recognition** | Identifying/understanding what the problem actually is — its inputs, outputs, constraints, success criteria — before solving |
| **Decomposition** | Breaking a complex problem into smaller, manageable **sub-problems** (different sub-tasks) |
| **Divide and conquer** | Repeatedly split a problem into **smaller instances of the same problem**, solve parts, **combine** results (e.g. binary search, merge/quicksort) — pairs with recursion |
| **Abstraction** | Removing/hiding unnecessary detail. **Representational** (drop irrelevant detail, e.g. Tube map) · **by generalisation** (group by shared traits) |

## Named methods (one-line use each)

| Method | What it is / use |
|---|---|
| **Backtracking** | Build a solution step by step; on a **dead end**, go back to the last decision and try another option (mazes, N-Queens, Sudoku; often recursive) |
| **Data mining** | Search **large data sets** for hidden patterns/correlations/trends (loyalty-card basket analysis, fraud detection) |
| **Heuristics** | A **rule of thumb** giving a **good-enough (not guaranteed optimal)** answer quickly when exact is too slow/impossible (satnav A*, Travelling Salesman) |
| **Performance modelling** | Use a **model** to predict/test how a system behaves before/instead of building it (server load, aircraft systems, algorithm scaling) |
| **Pipelining** | Split a process into **stages** so different stages work on **different items simultaneously**, raising throughput (CPU fetch–decode–execute) |
| **Visualisation** | Represent data/problems **graphically** (charts, maps, heat maps) to reveal patterns and aid decisions |

## Must-know definitions

| Term | Definition |
|---|---|
| **Sequence** | Instructions executed one after another, in order |
| **Selection** | Choosing between paths based on a condition |
| **Count-controlled iteration** | Loop repeating a fixed, known number of times (`for`) |
| **Condition-controlled iteration** | Loop repeating until a condition changes (`while`, `do...until`) |
| **Recursion** | A subroutine that calls itself, with a base case and a recursive case |
| **Base case** | The condition that stops recursion (no further recursive call) |
| **Call stack** | Stack of frames holding parameters, locals and return addresses for active calls |
| **Stack overflow** | Error from too many pushed frames (e.g. missing base case) |
| **Scope** | The region of a program where a variable can be accessed |
| **Local variable** | Accessible only within the subroutine that declares it |
| **Global variable** | Accessible throughout the whole program |
| **Modularity** | Splitting a program into self-contained subroutines |
| **Procedure** | Subroutine that performs a task but returns no value |
| **Function** | Subroutine that performs a task and **returns a value** |
| **Parameter** | Variable in a subroutine definition that receives an argument |
| **By value** | A copy is passed; original unaffected |
| **By reference** | The memory address is passed; original can be changed |
| **IDE** | One environment giving editor, debugger, translator and run tools |
| **Class** | Template defining the attributes and methods of its objects |
| **Object** | A specific instance of a class |
| **Encapsulation** | Bundling data + methods and keeping attributes private (data hiding) |
| **Inheritance** | A subclass acquiring attributes/methods of a superclass |
| **Polymorphism** | The same method call behaving differently for different object types |
| **Abstraction** | Removing/hiding unnecessary detail to focus on what matters |
| **Decomposition** | Breaking a problem into smaller sub-problems |
| **Divide and conquer** | Repeatedly splitting a problem, solving parts, combining results |
| **Heuristic** | A rule of thumb giving a good-enough answer quickly |

### ⚠️ Top exam reminders

- **Recursion:** always say it **calls itself** AND needs a **base case**; "why it fails" = **stack overflow** from a missing/unreached base case. Trace by showing frames **pushed then popped** in **LIFO** order.
- **By value vs by reference:** mark is the *consequence* — by value the original is **unchanged**; by reference it **can be changed** — not just "a copy is made".
- **Function vs procedure:** the defining difference is a **function returns a value**, a procedure does not.
- **OOP:** keep **encapsulation** (private data + public methods) ≠ **inheritance** (reuse parent) ≠ **polymorphism** (same call, different behaviour) distinct.
- **Computational methods:** give a **named example** AND say **why** the method suits the problem; for **heuristic** stress *good enough, not guaranteed optimal*.